

PADRÃO DE PROJETO COMPORTAMENTAL

STRATEGY (GOF)

por Marcelo Luis Rossi

O QUE É O PADRÃO STRATEGY?

- Ele permite:
 - Definir uma família de algoritmos;
 - Encapsular em classes separadas;
 - Tornar **intercambiável**.

PROJEÇÃO DO CÓDIGO

- SUBSTITUÍVEL
- COMPILAÇÃO E EXECUÇÃO

SEM

- QUEBRAR O SISTEMA
- ALTERAR A ESTRUTURA

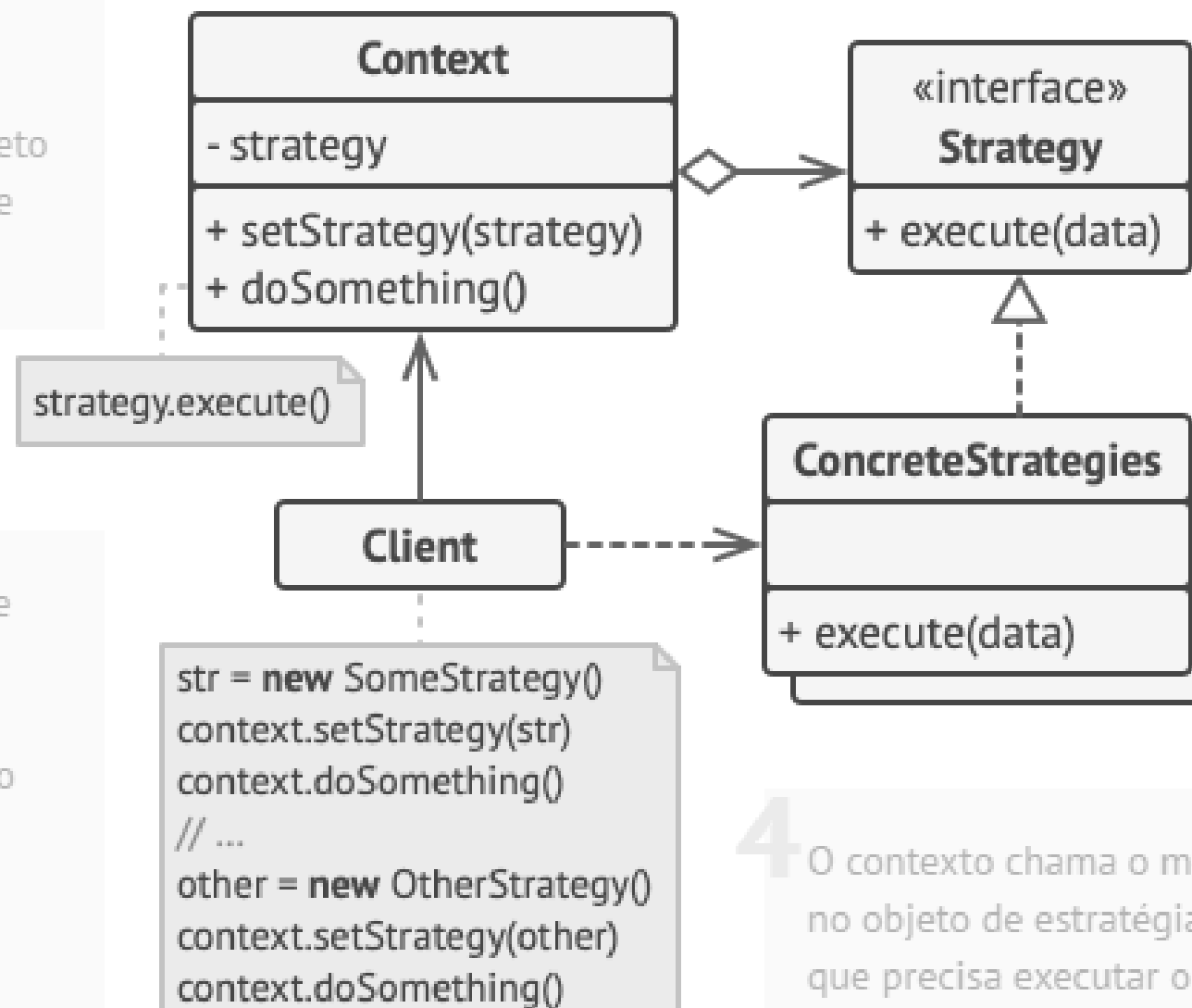
SOLUÇÃO E ESTRUTURA UML

Delega o comportamento para objetos separados (estratégias)

- **Context (Contexto):** A classe principal que o cliente interage. Mantém referência para a interface da Estratégia e não sabe como o algoritmo é executado.
- **Strategy (Estratégia):** Interface que declara o método comum a todos os algoritmos suportados.
- **Concrete Strategies (Estratégias Concretas):** Classes que implementam a interface Strategy, contendo a lógica específica de cada algoritmo.

ESTRUTURA UML

1 O **Contexto** mantém uma referência a uma das estratégias concretas e comunica-se com esse objeto apenas através da interface de estratégia.



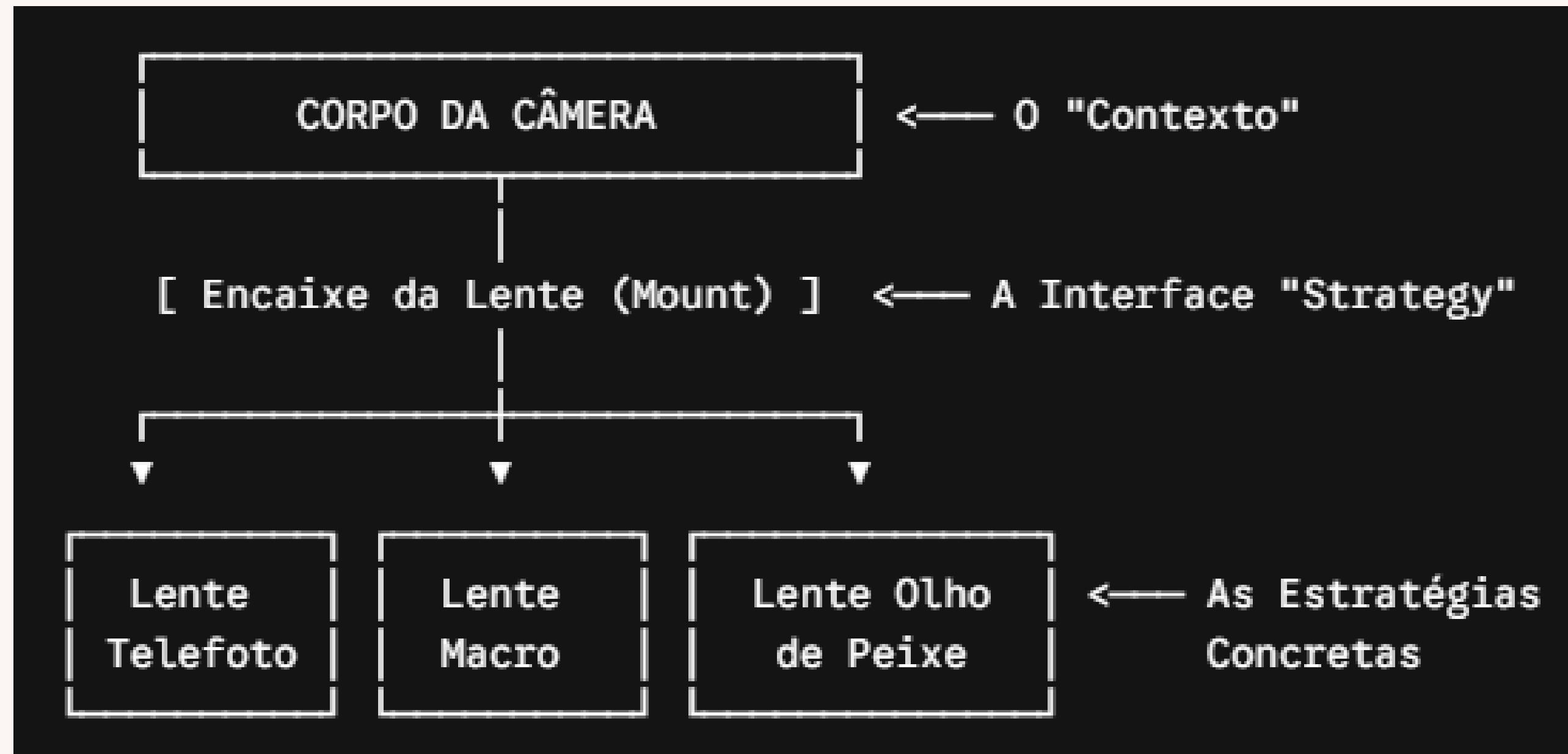
2 A interface **Strategy** é comum a todas as estratégias concretas. Ela declara um método que o contexto usa para executar uma estratégia.

3 As estratégias concretas implementam diferentes variações de um algoritmo, de acordo com o contexto.

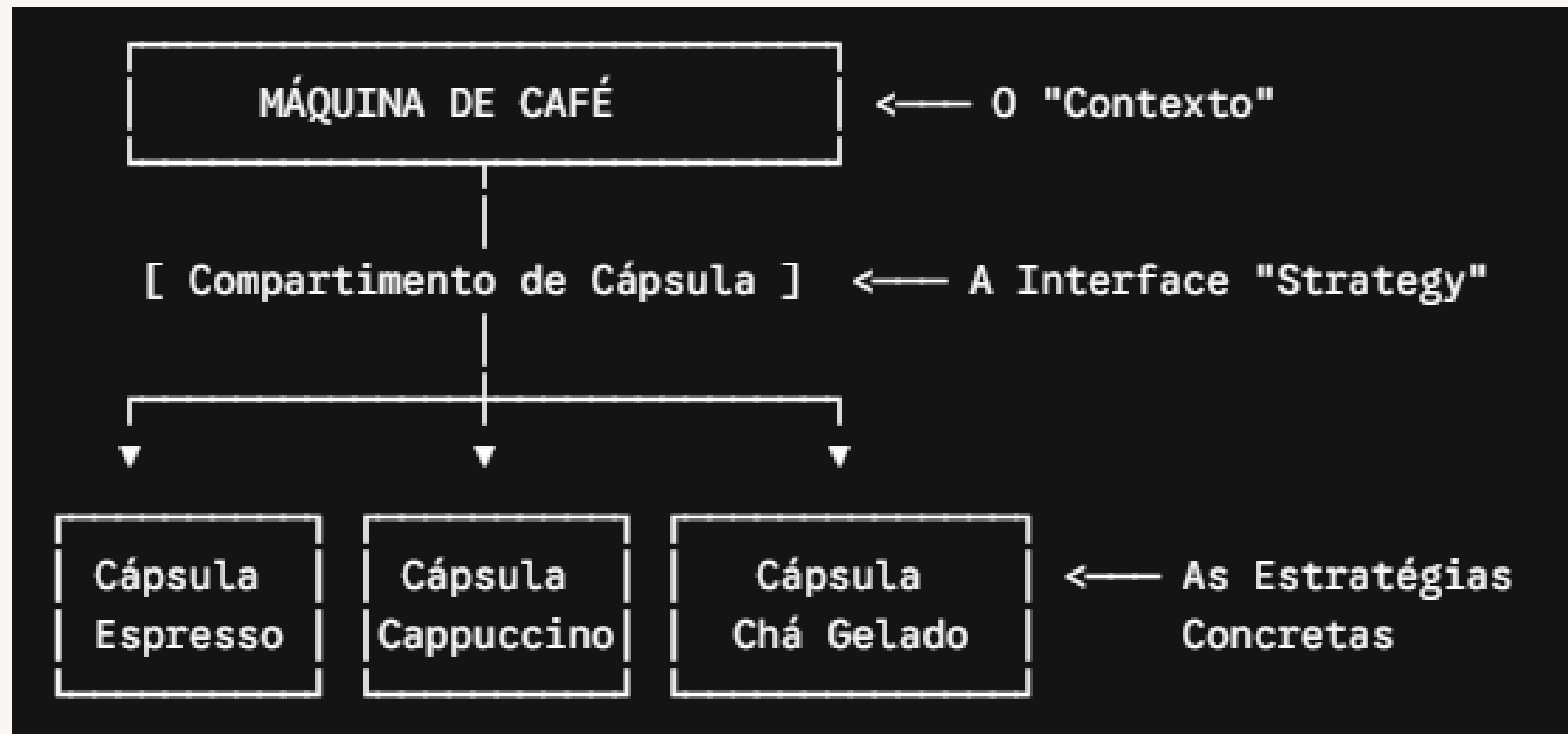
4 O contexto chama o método de execução no objeto de estratégia vinculado sempre que precisa executar o algoritmo. O contexto não sabe com que tipo de estratégia está trabalhando nem como o algoritmo é executado.

5 O **cliente** cria um objeto de estratégia específico e o passa para o contexto. O contexto expõe um método setter que permite aos clientes substituir a estratégia associada ao contexto em tempo de execução.

ANALOGIA



ANALOGIA



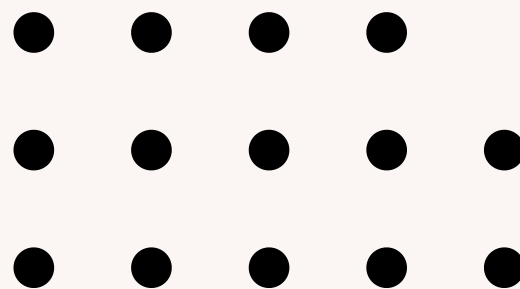
EXEMPLO – SEM STRATEGY

```
class CalcularFrete:
    def calcular(self, tipo: str, peso: float):
        if tipo == "SEDEX":
            return peso * 10.0
        elif tipo == "PAC":
            return peso * 5.0
```

- Variação: modificar a classe - risco de erros no código que já funcionava.
- Acoplamento: classe dependente de toda lógica do negócio.

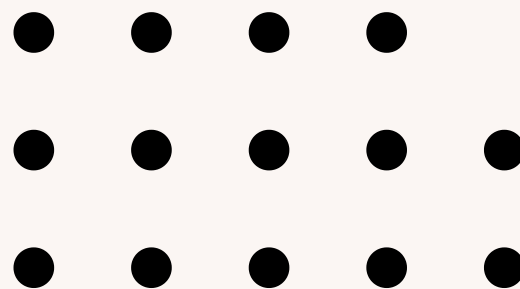
O QUE ELE PROCURA RESOLVER?

- Sistemas que costumam concentrar todo o código em uma única classe.
- Com muitas condicionais (if/else. switch/case);
- Que necessitam de variação;



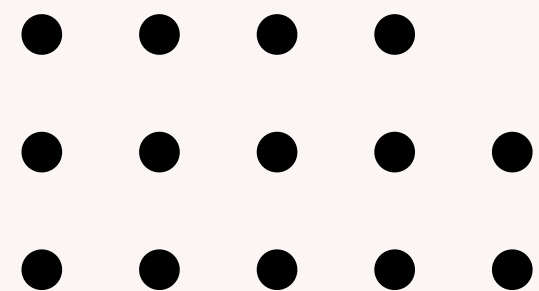
EM QUE MOMENTO APLICAR?

- Se ocorre a criação de novas regras → necessário abrir o mesmo arquivo e codar nele.
- Dificuldade em fazer testes unitários → se alterar regra a A, tem que alterar B, C e D junto.



QUANDO NÃO X UTILIZAR

- Quando o código possuir apenas uma ou duas regras e sem variações ou alteração.
- Se o comportamento do código **for simples**, sem lógica de negócio diferente.



CRIAÇÃO DA INTERFACE

```
from abc import ABC, abstractmethod

# 1. DEFINIR A FAMÍLIA / INTERFACE (Contrato comum)
class EstrategiaFrete(ABC):

    @abstractmethod
    def calcular(self, peso: float) -> float:
        pass
```

- Classe Abstrata (ABC): Impede a criação direta de um objeto `EstrategiaFrete`.
- Método Abstrato: Todas as classes filhas são obrigadas a implementar o método `calcular()`.

CRIAÇÃO DAS ESTRATÉGIAS

```
# 2. ENCAPSULAR EM CLASSES SEPARADAS (Estratégias Concretas)
class FreteSedex(EstrategiaFrete):

    def calcular(self, peso: float) -> float:
        return peso * 10.0

class FretePac(EstrategiaFrete):

    def calcular(self, peso: float) -> float:
        return peso * 5.0
```

- Herança: As classes FreteSedex e FretePac herdam de EstrategiaFrete.
- Isolamento de Regras: cada modalidade tem sua própria classe.
- Cumprem o método: *calcular()*, com suas regras e valores.

CRIAÇÃO DO CONTEXTO

```
# 3. TORNAR INTERCAMBIÁVEL (O Contexto)
```

```
class CalculadoraFrete:
```

```
    def __init__(self, estrategia: EstrategiaFrete):
```

```
        self.estrategia = estrategia # Pode ser substituída facilmente!
```

```
    def executar_calculo(self, peso: float) -> float:
```

```
        return self.estrategia.calcular(peso)
```

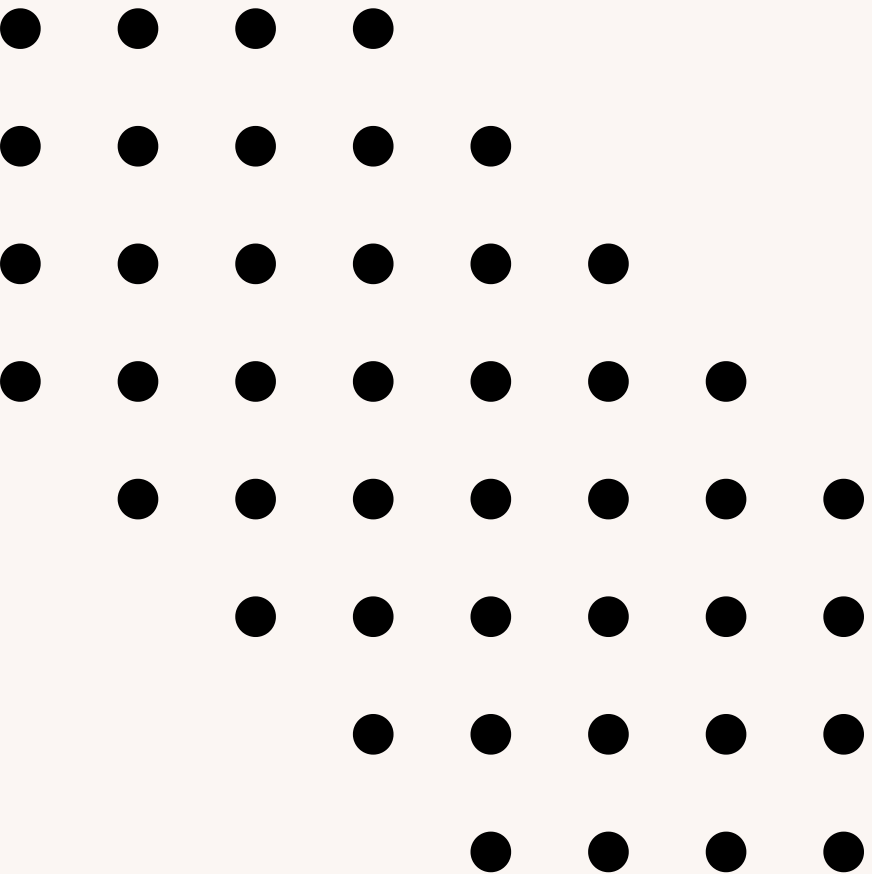
- Papel do Contexto: interação com o sistema para calcular o frete.
- Delegação de Tarefa: não calcula por conta própria, só repassa a responsabilidade.
- Polimorfismo: aceita qualquer classe que siga o contrato `EstrategiaFrete`.

EXECUÇÃO

```
# EXECUÇÃO
# Trocamos o comportamento apenas substituindo a classe, sem alterar a Calculadora!
calculadora = CalculadoraFrete(FreteSedex())
print(calculadora.executar_calculo(2.0)) # Saída: 20.0

# Intercambiável:
calculadora.estrategia = FretePac()
print(calculadora.executar_calculo(2.0)) # Saída: 10.0
```

- Papel do Cliente: É o código que inicializa o contexto e decide qual estratégia será utilizada.
- Troca de Comportamento: Ao alterar a estratégia para *FretePac()*, o comportamento da calculadora muda sem tocar na classe *CalculadoraFrete*.



OBRIGADO !

